

Name Source Generator Documentation

Version: 0.1.0 (In Development)

Language: Pure C

Namegen is a command-line application designed to generate **authentic Finnish names** from specific historical periods using decade-separated CSV data.

I. Key Features and Usage

Namegen provides a robust solution for generating historical names via the command line.

- **Historical Dataset:** Reads separate CSV files for **first, middle, and last names** for both male and female genders, separated by decades.
- **Gender Selection:** Supports gender selection (-g male / -g female), which directs the choice of first and middle names from the correct lists.
- **Time Period Selection:** A command-line parameter (-p <number>) allows names to be automatically generated based on a selected historical period.
- **Randomized Structure:** Middle name inclusion is **randomized (50% probability)**.
- **Command Line Interface (CLI):** Supports standard CLI flags (-v, -h, -V) with robust error handling.

II. Data Structures (Structs)

The program utilizes structures designed for dynamic memory management to store and process names.

Structure	Description	Key Fields	Purpose
NameList	A simple list for name strings.	char **names (name array), int count (number of names)	Used to store last names and individual columns from CSV files.
DecadeData	The main container for decade-separated CSV data.	char **decades (headers), NameList *lists (array of NameList structs), int num_decades (number of columns/decades)	Used to store first and middle names, where each column represents one decade.



III. Name Loading and Memory Management

The program includes two primary loading functions for different file formats, and memory management is handled by dedicated cleanup functions.

Loading Functions

- **load_names_multi_column(const char *filename, DecadeData *data)**
 - **Usage:** Loads multi-column CSV files (first and middle names).
 - **Operation:** Reads the header row first and creates a new NameList struct for each header (DecadeData.lists). Then, reads row by row, adding names to the correct decade list based on the column.
- **load_names_simple(const char *filename, NameList *list)**
 - **Usage:** Loads simple text files (one name per line) or only the first column of a CSV (last names).
 - **Operation:** Reads the file line by line and saves each name directly to the given NameList structure.

Memory Management

Memory is **dynamically allocated** (malloc, realloc, strdup) for every loaded name and data structure. Therefore, memory must be freed:

- **void free_names(NameList *list)**
 - **void free_decade_data(DecadeData *data)**
 - These functions iterate through every allocated string and free its memory (free()), free the pointer lists, and finally reset the counters to prevent **memory leaks**.
-

IV. Main Program (main) Logic

Step	Operation	Purpose
1. Initialization	setlocale(), srand(), Windows fixes	Sets UTF-8 localization and the random number generator seed.
2. Data Loading	Calls to load_names_multi_column and load_names_simple	Loads all first, middle, and last names (male and female) into memory.
3. Command Line	Processes flags (-v, -p, -g, -n)	Sets the variables that control generation (e.g., period_index, gender_flag, name_count).
4. Checks	if (first_names.num_decades == 0 ...)	Critical step: Checks that the required data files have been loaded.
5. Name Generation	Loop (for (k=0; k < name_count; k++)) and rand()	Selects: a) a first name from the selected decade, b) a last name from the complete list, and c) a middle name with a 50% probability.
6. Memory Release	free_decade_data(), free_names()	Frees all dynamically allocated memory before the program exits.

V. TODO / Next Steps

Goal	Description	Status
Output Control	Ability to select how many names are generated at once (e.g., 1 or 10) using a new flag (-n or --count).	Next Task
Setting Flexibility	Allow the middle name probability to be adjusted (e.g., -m 80 for 80% probability).	Planned
Genesim Integration	Preparation for use by the genesim project (library module).	Planned

Now that the documentation is updated and translated, would you like to proceed with implementing the next task: **Output Control** (-n <lk>)?